Bulkification:
The Process of writing apex code to handle multiple records at once rather than a single record at a time .

For example, this is bad ❌:

```
for (Customer__c customer : Trigger.new) {

    List<Membership__c> memberships = [
        SELECT Id
        FROM Membership__c
        WHERE Customer__c = :customer.Id
    ];
}
```

Why? The **SOQL query is inside the loop**.

If Salesforce processes:

200 Customers

↓

Loop runs 200 times

↓

200 SOQL queries ❌

↓

Governor Limit error

Instead, bulkified code follows:

Trigger.new

↓

Collect IDs into Set

↓

ONE SOQL query

↓

Store results in List/Map

↓

Process records

↓

ONE DML

Example:

```
// STEP 1: Create Set for Customer IDs
Set<Id> customerIds = new Set<Id>();
// STEP 2: Collect all Customer IDs
for (Customer__c customer : Trigger.new) {

    customerIds.add(customer.Id);
}
// STEP 3: ONE SOQL for all Customers
List<Membership__c> memberships = [
    SELECT Id, Customer__c
```

```
    FROM Membership__c
    WHERE Customer__c IN :customerIds
];


// STEP 4: Process the results
for (Membership__c membership : memberships) {

    System.debug(
        'Membership = ' + membership.Id
    );
}
```

Bulkification = collect first, query once, process many.

Why `Set<Id>` instead of `List<Id>`?

Both can store multiple IDs, but **Set** automatically removes duplicates.

`List<Id>`

```
List<Id> customerIds = new List<Id>();
customerIds.add(customer1.Id);
customerIds.add(customer1.Id);
```

customerIds.add(customer2.Id);

It can contain:

C001

C001 ← **duplicate**

C002

`Set<Id>` ✅

Set<Id> customerIds = new Set<Id>();

customerIds.add(customer1.Id);

customerIds.add(customer1.Id);

customerIds.add(customer2.Id);

Result:

C001

C002

The duplicate `C001` is automatically removed.

Why useful in bulkification?

Suppose 200 records reference customers:

Membership 1 → Customer A

Membership 2 → Customer A

Membership 3 → Customer B

Membership 4 → Customer A

Membership 5 → Customer C

We don't need:

A, A, B, A, C ❌

We need only:

A, B, C ✅

So we use:

Set<Id> customerIds = new Set<Id>();

then one SOQL:

```
List<Customer__c> customers = [
    SELECT Id, Name
    FROM Customer__c
    WHERE Id IN :customerIds
];
```

So remember:

List → multiple values, duplicates allowed

Set → multiple UNIQUE values

Map → Key → Value

Why we use List in the SOQL instead of Set?

`List<Customer__c>` ✅

```
List<Customer__c> customers = [
    SELECT Id, Name
    FROM Customer__c
];
```

A List stores records in a collection:

[Customer A, Customer B, Customer C]

You can access by position:

customers[0]

customers[1]

So List is convenient when you want to loop through all returned records

`Set<Customer__c>`

A Set is designed around uniqueness, not position.

Conceptually:

{Customer A, Customer B, Customer C}

There is no:

customers[0]

because a Set has no index/position.

A Set is useful when your main requirement is:

"I need unique values."

For example:

Set<Id> customerIds = new Set<Id>();

This is perfect because:

Customer A

Customer A

Customer B

Customer C

becomes:

Customer A

Customer B

Customer C

So our bulkification pattern becomes:

Trigger.new

↓

Set<Id>

(unique IDs)

↓

ONE SOQL

↓

List<Customer__c>

(all returned records)

↓

Map<Id, Customer__c>

(fast lookup when needed)

Why do we need a Map?

Suppose we queried 100 Customers:

```
List<Customer__c> customers = [
    SELECT Id, Name
    FROM Customer__c
```

];

If you want to find a particular Customer by ID, a Map is much cleaner.

Create a Map

Map<Id, Customer__c> customersById =

new Map<Id, Customer__c>(customers);

Now the data looks conceptually like:

Customer ID Customer Record

---------- ---------------

C001 → Vaibhav

C002 → Rahul

C003 → Krishna

Get one Customer

Customer__c customer =

customersById.get(customerId);

`.get()` means:

Give me the Customer whose ID is `customerId`.

Small example

List<Customer__c> customers = [

SELECT Id, Name

```
    FROM Customer__c
];

Map<Id, Customer__c> customersById =
    new Map<Id, Customer__c>(customers);

for (Customer__c customer : customers) {

    Customer__c sameCustomer =
        customersById.get(customer.Id);

    System.debug(sameCustomer.Name);
}
```

Why Map is useful in triggers ⭐

Suppose:

200 Memberships

↓

200 Customer IDs

↓

Set<Id>

↓

ONE SOQL

↓

Customer records

↓

Map<Id, Customer__c>

↓

Find Customer instantly by ID

Set + SOQL + List + Map

```
//
==========================================================
// STEP 1: Create a Set
//
==========================================================


// Set stores UNIQUE Customer IDs.
Set<Id> customerIds = new Set<Id>();
```

```
//
==========================================================
// STEP 2: Collect Customer IDs
//
==========================================================

for (Membership__c membership : Trigger.new) {

    if (membership.Customer__c != null) {

        customerIds.add(
            membership.Customer__c
        );
    }
}



//
==========================================================
// STEP 3: ONE SOQL QUERY
//
==========================================================
```

```
// List stores the Customer records returned by SOQL.
List<Customer__c> customers = [

    SELECT Id, Name, Membership_Status__c

    FROM Customer__c

    WHERE Id IN :customerIds
];



//
==========================================================
// STEP 4: Convert List → Map
//
==========================================================

// Map:
// Customer ID → Customer Record
Map<Id, Customer__c> customersById =
```

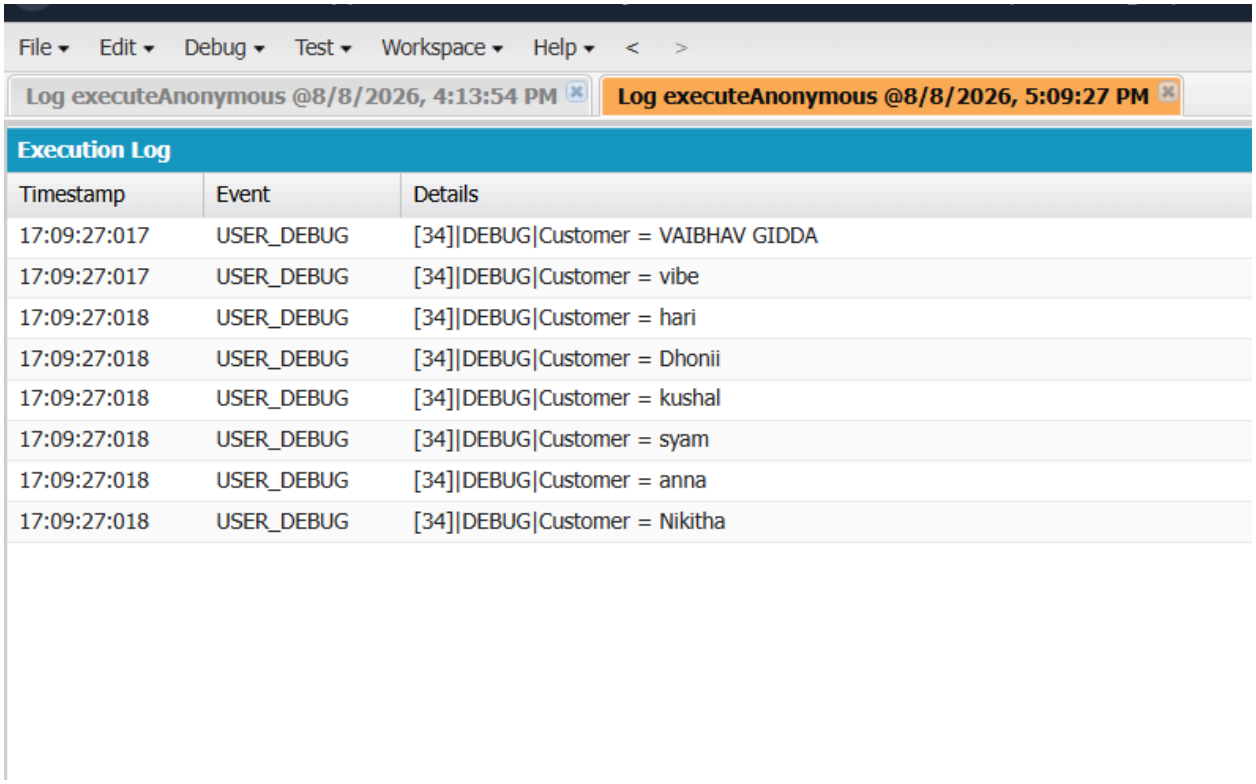
```
    new Map<Id, Customer__c>(customers);



//
==============================================================
// STEP 5: Use the Map
//
==============================================================
for (Membership__c membership : Trigger.new) {

    if (membership.Customer__c != null) {

        Customer__c customer =
            customersById.get(
                membership.Customer__c
            );
        System.debug(
            'Customer Name = ' + customer.Name
        );
    }
}
```



Membership records

↓

↓

Set<Id>

"Give me unique Customer IDs"

↓

↓

ONE SOQL

↓

↓

List<Customer__c>

"Here are the Customers"

↓

↓

Map<Id, Customer__c>

"ID → Customer"

↓

↓

.get(CustomerId)

↓

Get the Customer quickly